Studio Nx

JS 109 — Transport 위치 제어

2026-10-14

JS 109 — `Transport` 위치 제어 (`pause` · `resume` · `seek`)

항목	내용
파일	`studionx/frontend/static/js/audio/scheduler.js` (같은 클래스에 메서드 추가)
작업	S1-05 오디오 엔진 기초: 여섯 번째 함수 (FR-201 재생/정지/일시정지/위치 이동)
작성	Claude (AI), 2026-10-14, 사용자 검토
브랜치	`feature/phase1/s1-05-audio-engine`
다음	재생 중 템포 변경(`setTempo`), 그다음 메트로놈, 루프

1. 하는 일

108의 `Transport` 에 일시정지·이어서·위치 이동을 붙였습니다. 겸해서 `start`·`stop`·`positionTick` 이 쓰던 공통 부분을 내부 메서드(`_clearTimer`·`_anchor`·`_alignUp`·`_scheduleTimer`)로 묶었습니다(108 코드를 이렇게 다듬었습니다).

메서드	하는 일
`pause()`	지금 위치(`positionTick`)를 기억하고 타이머를 끕니다. 이후 `positionTick` 은 그 위치에 멈춥니다
`resume()`	멈춘 위치를 다음 단계 경계로 올려(최대 한 단계) 그곳부터 다시 예약합니다
`seek(tick)`	그 위치로 옮깁니다. 재생 중이면 그 위치에서 다시 예약하고, 멈춰 있으면 위치만 옮깁니다

`resume` 이 경계로 올리는 이유는, 멈추기 직전에 이미 예약된 단계와 겹치지 않게 하기 위해서입니다.

2. 코드

```
  pause() {
    if (!this.playing) return this;
    const tick = this.positionTick();
    this._clearTimer();
    this.playing = false;
    this._anchor(tick, this.clock.now());
    return this;
  }

  resume() {
    if (this.playing) return this;
    const from = this._alignUp(this.cursorTick);
    this.playing = true;
    this._anchor(from, this.clock.now() + this.startDelay);
    this.pump();
    this._scheduleTimer();
    return this;
  }

  seek(tick) {
    const wasPlaying = this.playing;
    this._clearTimer();
```

```
    this._anchor(tick, wasPlaying ? this.clock.now() + this.startDelay : this.clock.now());
    if (wasPlaying) {
      this.pump();
      this._scheduleTimer();
    }
    return this;
  }

  positionTick() {
    if (!this.playing) return this.originTick;
    const originSeconds = ticksToSecondsWithChanges(this.originTick, this.baseBpm, this.changes);
    const nowSeconds = originSeconds + (this.clock.now() - this.originTime);
    return secondsToTicks(nowSeconds, this.baseBpm, this.changes);
  }

  _clearTimer() {
    if (this.timerId != null) { this.clearTimer(this.timerId); this.timerId = null; }
  }
  _anchor(tick, time) {
    this.originTick = tick; this.originTime = time;
    this.cursorTick = tick; this.cursorTime = time;
  }
  _alignUp(tick) { return Math.ceil(tick / this.stepTicks) * this.stepTicks; }
  _scheduleTimer() {
    const self = this;
    this.timerId = this.setTimer(function () { self.pump(); }, this.intervalMs);
  }
```

최신 문법으로 쓰면 (공부용)

```
  positionTick() {
    if (!this.playing) return this.originTick;
    const originSeconds = ticksToSecondsWithChanges(this.originTick, this.baseBpm, this.changes);
    return secondsToTicks(originSeconds + this.clock.now() - this.originTime, this.baseBpm, this.change
s);
  }
```

최신 문법	실제 코드	뜻
비공개 필드 `#timerId`	`_timerId`(밑줄 이름)	진짜 비공개는 `#` 이지만, 전통 문법을 지키려고 밑줄 이름을 씁니다
`Math.ceil`	(같음)	실제 코드도 `Math.ceil` 을 씁니다

3. 왜 이렇게

선택	이유
`pause` 는 위치를 기억하고 멈춤	`positionTick` 이 `!playing` 일 때 `originTick` 을 돌려주므로, 시계가 흘러도 일시정지 중에는 재생선이 그 자리에 멈춥니다
`resume` 은 다음 단계 경계로 올림	멈추기 직전에 예약된 단계와 겹쳐 소리가 두 번 나는 것을 줄입니다(최대 한 단계 = 1/16박)
`seek` 는 재생 중일 때만 다시 예약	멈춰 있을 때는 위치만 옮기고 다음 `start`·`resume` 이 그 위치를 씁니다

선택	이유
이미 잡힌 소리 창은 남음	`stop`·`pause` 는 타이머만 끊습니다. 최대 lookahead(0.1초) 안에서 이미 예약된 소리는 울린 뒤 멈춥니다. 곧바로 끊으려면 소리를 잇는 함수에서 그 노드를 취소해야 합니다(문서에 남김)
`seek`·재생 시작에 `startDelay` 를 줌	지난 시각에 예약하는 것을 막습니다. 그래서 위치가 `originTick` 에 정확히 닿는 시각은 `startDelay` 뒤입니다(재생선이 아주 조금 앞서 보일 수 있음)
공통 부분을 내부 메서드로 묶음	`start`·`stop`·`pause`·`resume`·`seek` 이 모두 타이머 정리와 기준점 맞추기를 써서, 한 곳에 모아 실수를 줄였습니다

4. 시험 (Node에서 직접 실행, 21개 통과 · 가짜 시계·가짜 타이머)

확인	결과
일시정지: `playing` 거짓, 타이머 꺼짐, 시계가 흘러도 위치 고정	✓
이어서: 타이머 켜짐, 단계 경계에 맞춰 예약, 이후 tick이 240씩 이어짐	✓
`resume`·`pause` 두 번 = 아무 일 없음 / 타이머 한 번만 정리	✓
`seek` (멈춤): 위치만 이동, 예약 안 함	✓
`seek` (재생 중): 새 위치에서 예약, 이후 240씩 이어짐	✓
`stop` 후 멈춰 있고 위치 유지	✓
`start` 다시 = 처음 위치부터 낮은 tick 으로 예약	✓